

요구사항 정의서

서비스 목표, 사용자 요구사항, 시스템/비기능 요구사항, 데모 성공 기준

프로젝트

YumYum

기준 커밋

a963b9d

생성일

2026. 06. 25.

1. 프로젝트 개요

YumYum은 사용자의 냉장고 재고를 기반으로 유통기한을 관리하고, 식단 기록과 레시피 추천을 연결하는 식생활 관리 서비스이다. 사용자는 OAuth 로그인 후 온보딩을 완료하고, 냉장고 재고를 수기/OCR/바코드 방식으로 등록한다. 서비스는 마감 임박 재료를 알려주고, 보유 재료를 활용한 레시피를 추천하며, 식단 기록을 통해 일일 영양 요약を提供한다.

2. 핵심 목표

- 냉장고 재고를 빠르게 등록하고 유통기한을 놓치지 않게 한다.
- 보유 재료 중심의 레시피 추천으로 음식물 낭비를 줄인다.
- 식단 기록과 영양 요약을 통해 사용자가 섭취 현황을 확인하게 한다.
- 관리자 화면에서 공지, 배치, API 사용량을 확인해 운영 안정성을 높인다.
- 외부 API는 `mock/live/fallback` 모드로 분리해 데모 환경에서도 중단 없이 동작하게 한다.

3. 사용자와 문제 정의

사용자	문제	YumYum 해결 방식
1인 가구/자취생	냉장고에 무엇이 남았는지 기억하기 어렵다.	재고 목록과 유통기한 상태를 대시보드에서 요약한다.
바쁜 학습자/직장인	식재료를 사두고도 활용하지 못해 버리는 일이 생긴다.	마감 임박 재료를 알림과 추천 레시피에 우선 반영한다.
식단을 기록하는 사용자	먹은 음식과 보유 식재료 데이터가 분리되어 있다.	식단 기록과 냉장고 재고 기반 추천을 한 서비스 흐름으로 연결한다.
운영자/PM	데모 중 외부 API 장애나 키 누락으로 시연이 멈출 수 있다.	<code>mock/live/fallback</code> 모드와 관리자 사용량 화면으로 안정성을 확보한다.

4. 사용자 요구사항

ID	요구사항	우선순위	수용 기준
UR-01	사용자는 Google/Naver/Kakao OAuth로 로그인할 수 있다.	High	로그인 후 신규 사용자는 온보딩 화면으로 이동한다.
UR-02	사용자는 성별, 생년월일, 키, 몸무게, 활동량 등 기본 정보를 입력할 수 있다.	High	온보딩 완료 후 대시보드에 접근할 수 있다.
UR-03	사용자는 냉장고 재고를 수기로 여러 개 등록할 수 있다.	High	등록 후 목록과 상세 화면에서 확인할 수 있다.
UR-04	사용자는 영수증/OCR 또는 바코드 이미지로 재고 후보를 분석할 수 있다.	Medium	분석 결과가 다중 입력 폼에 매핑된다.
UR-05	사용자는 재고 목록을 조회, 수정, 삭제할 수 있다.	High	변경 결과가 목록과 대시보드 통계에 반영된다.
UR-06	사용자는 음식명을 검색해 식단 기록에 추가할 수 있다.	High	저장 후 일일 요약 카드와 기록 목록이 갱신된다.
UR-07	사용자는 보유 재료 기반 레시피 추천을 받을 수 있다.	High	mock 모드에서도 추천 결과가 표시되고 최근 추천 이력이 저장된다.
UR-08	사용자는 유통기한/공지 알림을 조회하고 읽음 처리할 수 있다.	Medium	읽음 처리 후 안 읽은 알림 수가 갱신된다.
UR-09	관리자는 로그인 후 대시보드, API 사용량, 배치, 공지, 감사 로그를 확인할 수 있다.	Medium	<code>/admin</code> 에서 각 탭이 표시되고 주요 조작이 가능하다.

5. 시스템 요구사항

ID	요구사항	수용 기준
SR-01	FrontEnd, BackEnd, MySQL, OCR 서비스를 Docker Compose로 실행한다.	<code>docker compose up --build</code> 기준 각 서비스가 기동된다.
SR-02	민감값은 Git에 커밋하지 않고 환경변수로 주입한다.	<code>application.yml</code> 은 비밀값 없는 기본값만 포함한다.
SR-03	외부 AI/OCR/식품안전나라 API는 통합 모드로 제어한다.	<code>INTEGRATION_MODE=mock/live/fallback</code> 을 지원한다.
SR-04	백엔드는 공통 응답 포맷과 예외 처리를 사용한다.	성공/실패 응답이 일관된 형태로 내려간다.
SR-05	인증 요청은 JWT 기반으로 보호한다.	인증 실패 시 401/403이 반환되고 FE는 세션 만료를 처리한다.

6. 비기능 요구사항

- 데모 안정성: 외부 API 장애 또는 키 미설정 상황에서도 mock/fallback 결과로 핵심 화면을 시연할 수 있어야 한다.
- 사용성: 재고 등록, 식단 기록, 추천 생성 흐름은 한 화면에서 이어져야 한다.
- 유지보수성: 도메인별 controller/service/dao/entity/dto 구조를 유지한다.
- 확장성: 외부 API 호출은 provider/client 계층으로 분리해 실제 API와 mock 구현을 교체할 수 있어야 한다.
- 보안: OAuth client secret, API key, JWT secret은 문서/코드/캡처에 노출하지 않는다.

7. 핵심 품질 기준

기준	설명	확인 방법
부팅 가능성	fresh clone과 Docker 환경에서 서비스가 기동되어야 한다.	<code>docker compose config --quiet</code> , Docker 실행 확인
빌드 가능성	FE/BE가 각각 빌드 또는 컴파일되어야 한다.	<code>npm run build</code> , <code>mvn -DskipTests compile</code>
발표 안정성	외부 API가 없어도 추천/OCR 흐름 설명과 화면 캡처가 가능해야 한다.	<code>INTEGRATION_MODE=mock</code> 기준 화면 캡처
개인정보 보호	캡처와 문서에 실제 이메일, 토큰, secret이 없어야 한다.	문서 검색과 시각 검수
사용 흐름 완결성	로그인부터 관리자 화면까지 발표 시나리오가 끊기지 않아야 한다.	수동 시연 체크리스트

8. 데모 성공 기준

- 1. 실제 OAuth 로그인 후 신규 사용자 온보딩을 완료한다.
- 2. 냉장고에 식재료를 수기 등록하고 목록/상세/수정/삭제를 확인한다.
- 3. 식단 기록에서 김밥 을 검색해 저장하고 일일 요약 변화를 확인한다.
- 4. 레시피 추천을 생성하고 결과를 레시피 화면에서 확인한다.
- 5. 알림 목록을 조회하고 읽음 처리한다.
- 6. 관리자 화면에서 대시보드, 공지, 배치, API 사용량 화면을 확인한다.

9. 범위와 제외 사항

구분	이번 제출 범위	이후 개선 범위
사용자	단일 사용자 데모 흐름	다중 사용자 동시 접속, 운영 모니터링
추천	mock/live/fallback provider 구조와 저장 이력	개인화 모델 고도화, 장기 추천 품질 평가
인증	OAuth 로그인과 JWT 기반 API 보호	HttpOnly cookie 전환, 장기 토큰 운영 정책
배포	Docker Compose 기반 로컬/데모 실행	공개 URL, HTTPS, OAuth redirect 운영값
테스트	빌드/컴파일/수동 시연 중심	E2E 자동화, 80% 이상 커버리지